



Company Filings Add Files — Technical Documentation

Coresight Research Portal — Technical Documentation

Document version: 1.0

Generated: June 29, 2026

Classification: Internal — Engineering Reference

CONFIDENTIAL — INTERNAL USE ONLY — DO NOT DISTRIBUTE

This document contains proprietary information belonging to Coresight Research. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Table of Contents

Company Filings Add Files — Technical Documentation	1
1. Purpose and scope	3
2. Environment and access guards	3
3. High-level architecture	4
4. UI sections (no tabs)	6
5. Session state	7
6. Configuration	8
7. Azure helpers	9
8. Upload flow	10
9. Delete flow	11
10. Browse / download flow	12
11. Audit logging — coreiq_azure_blob_audit_log	13
12. Components used	14
13. Relationship to Company Filings viewer	15
14. Suggested folder conventions (non-SEC)	15
15. Security considerations	16
16. Error handling patterns	16
17. Function index	17
18. Dependencies (Python packages)	18
19. Operational checklist	18

Application: Market Data Portal (MDP)

Page module: `app/pages/company_filings_add_files.py`

URL: `/company_filings_add_files`

Registered in: `app/main.py` as `( "pages/company_filings_add_files.py", "Add Files", "company_filings_add_files", False )`

UI title: Company Filings File Manager

1. Purpose and scope

This page is an **administrative Azure Data Lake Storage (ADLS Gen2) file manager** for company filing PDFs (primarily non-SEC / Capital Q documents). It allows authorized users to:

- Create folder hierarchies under a configured root prefix
- Upload files (always renamed to `filing.pdf`)
- Browse folder contents (list, download)
- Delete files or folders recursively (restricted role)
- Audit every upload/delete in MySQL
- Send SMTP email notifications on successful uploads

It complements `app/pages/company_filings.py`, which **reads** blobs the upload page **writes**. The filings viewer expects non-SEC paths like:

```
{TICKER}/{YEAR}/{annual-report|interim-report-Q*}/filing.pdf
```

2. Environment and access guards

2.1 Environment restriction

```
if config.env not in (Environment.LOCAL, Environment.STAGING):  
    st.switch_page("pages/home.py")  
    st.stop()
```

Production is blocked — this tool runs only in `LOCAL` and `STAGING`.

2.2 Authentication

- OIDC via `require_auth()` at page entry.

2.3 ACL model (`coreiq_page_access_control`)

Capability	Minimum permission	Constant
View page / browse / download	Authenticated (default)	—
Upload	<code>edit</code>	<code>AccessControlManager.check_access(..., "edit")</code>
Delete file/folder	<code>delete_admin</code>	<code>AccessControlManager.check_access(..., "delete_admin")</code>

```
CAN_UPLOAD = AccessControlManager.check_access("company_filings_add_files",
LOGGED_IN_EMAIL, "edit")
CAN_DELETE = AccessControlManager.check_access("company_filings_add_files",
LOGGED_IN_EMAIL, "delete_admin")
```

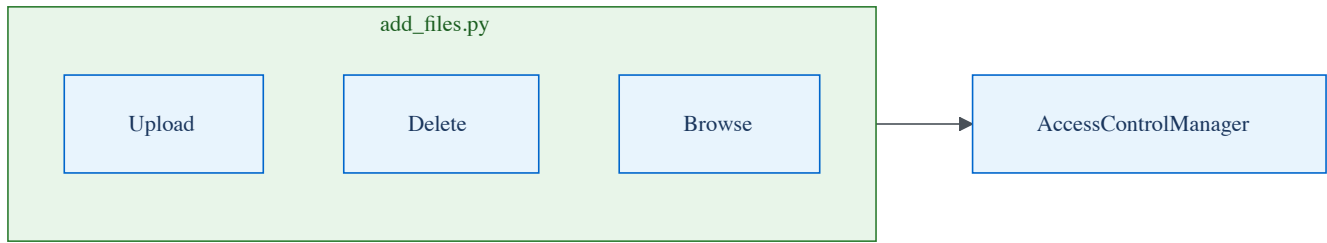
Note: Module docstring mentions `shashankgupta@coresight.com` for delete; actual enforcement is ACL-based `delete_admin`, not a hardcoded email.

3. High-level architecture

Admin upload page: ACL gate, Azure Blob storage, and audit logging for non-SEC filings.

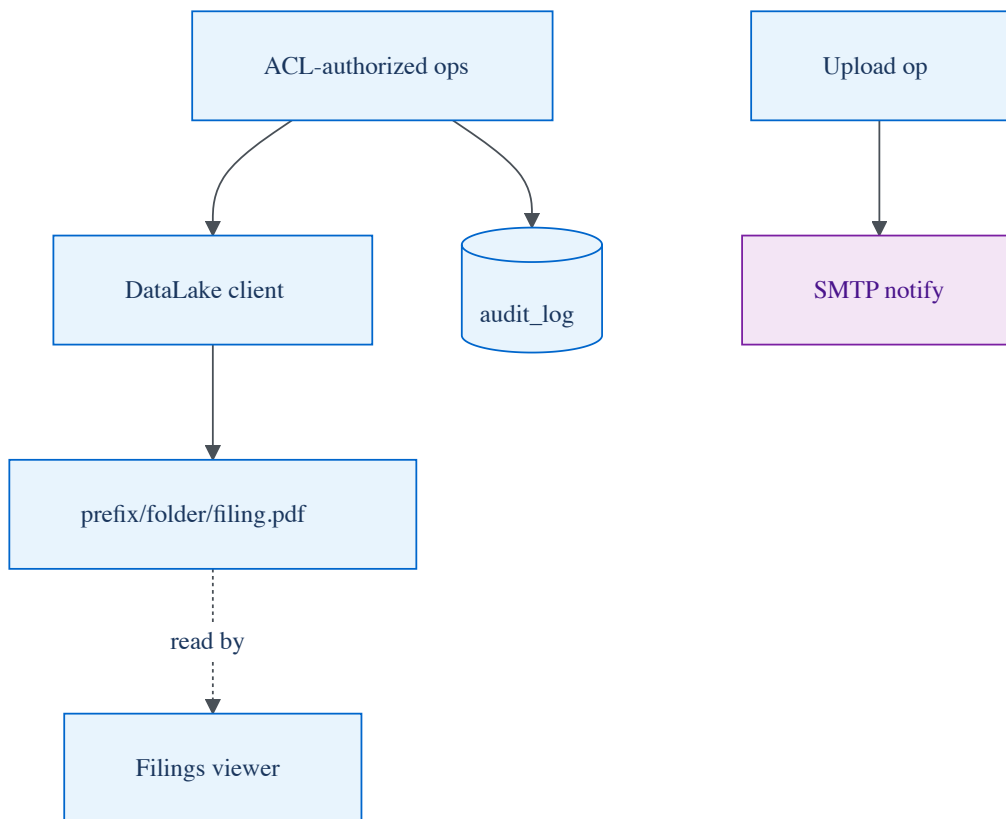
Architecture — Part A: UI and access control

Architecture — Part A: UI and access control



Architecture — Part B: Storage, audit, and viewer

Architecture — Part B: Storage, audit, and viewer



Important distinction: This page uses `azure.storage.filedatalake.DataLakeServiceClient` (hierarchical namespace / ADLS Gen2). `company_filings.py` uses `azure.storage.blob.BlobServiceClient`. Both target the same storage account/container env vars but different SDK surfaces.

4. UI sections (no tabs)

The page is a **single vertical layout** with dividers — not tabbed.

Section	Visible when	Controls
Current User Context (expander)	Always	Email, <code>CAN_UPLOAD</code> , <code>CAN_DELETE</code>
Target Location	Always	Folder path text input, disabled root prefix display, target path info
Upload Files	Always	File uploader, overwrite checkbox, primary upload button
Delete Folder	<code>CAN_DELETE</code> only	Folder path, type <code>DELETE</code> confirm, recursive delete button
Browse Existing Folder	Always	Browse path, "Show folder contents", per-item download/delete

4.1 Upload section details

Control	Key / behavior
Folder path input	Free text, e.g. <code>BRBY/2025/annual-report</code>
Root prefix	Read-only display of <code>AZURE_BLOB_PREFIX</code>
Target preview	<code>info</code> box: <code>{container}/{full_path}/filing.pdf</code>
File uploader	<code>accept_multiple_files=False</code> , disabled if <code>!CAN_UPLOAD</code>
Overwrite	Checkbox, default <code>True</code>
Upload button	"Create folder and upload file", <code>type="primary"</code>

Forced filename: All uploads saved as `FORCED_UPLOAD_FILENAME = "filing.pdf"` regardless of original name.

4.2 Delete section details

- Recursive directory delete via `delete_directory_recursive()`
- Requires typing `DELETE` exactly in confirmation field
- Hidden entirely when `CAN_DELETE` is false

4.3 Browse section details

Action	Permission	Behavior
Show folder contents	All	Sets <code>show_folder_contents=True</code> , stores <code>last_browse_full_path</code>
List items	All	Up to 300 entries, non-recursive
Download file	All	<code>download_file_bytes()</code> → <code>st.download_button</code>
Delete folder/file	<code>CAN_DELETE</code>	Per-row buttons, <code>st.rerun()</code> on success

5. Session state

Key	Type	Purpose
<code>show_folder_contents</code>	bool	Whether browse results are shown
<code>last_browse_full_path</code>	str	Full ADLS path after <code>build_full_path()</code>
<code>delete_folder_input</code>	str	Widget key for delete path
<code>delete_confirm_text</code>	str	DELETE confirmation
<code>browse_path</code>	str	Widget key for browse path

No ticker/doc_type session keys — paths are free-form strings.

6. Configuration

6.1 Environment variables

Variable	Default / notes
<code>AZURE_STORAGE_ACCOUNT_NAME</code>	Required
<code>AZURE_STORAGE_ACCOUNT_KEY</code>	Required
<code>AZURE_BLOB_CONTAINER</code>	<code>azure-storage-test</code> — ADLS filesystem name
<code>AZURE_BLOB_PREFIX</code>	Optional root prefix inside filesystem
<code>SMTP_SERVER</code>	<code>smtp.office365.com</code>
<code>SMTP_PORT</code>	<code>587</code>
<code>FROM_EMAIL</code>	<code>dataautomation@coresight.com</code>
<code>EMAIL_PASSWORD</code>	SMTP auth

Loaded via `load_dotenv(find_dotenv(usecwd=True))` plus `core.config.config` for DB.

6.2 Constants (in-module)

Constant	Value
<code>AUDIT_TABLE_NAME</code>	<code>coreiq_azure_blob_audit_log</code>
<code>FORCED_UPLOAD_FILENAME</code>	<code>filing.pdf</code>
<code>UPLOAD_SUCCESS_RECIPIENTS</code>	Fixed list of 4 Coresight emails

6.3 DB connection

`get_db_connection()` uses **PyMySQL** directly with `config.database.connect_args` (SSL/CA parity with main app). This page does **not** use `db_manager` singleton.

7. Azure helpers

7.1 Client initialization

```
account_url = f"https://{AZURE_STORAGE_ACCOUNT_NAME}.dfs.core.windows.net"
service_client = DataLakeServiceClient(account_url=account_url,
credential=AZURE_STORAGE_ACCOUNT_KEY)
filesystem_client = service_client.get_file_system_client(AZURE_BLOB_CONTAINER)
```

Endpoint uses `.dfs.core.windows.net` (Data Lake), not `.blob.core.windows.net`.

7.2 Path helpers

Function	Behavior
<code>normalize_path_part(value)</code>	Strip slashes
<code>build_full_path(folder_path)</code>	<code>{AZURE_BLOB_PREFIX}/{folder}</code> or folder alone
<code>ensure_directory_exists(fs, path)</code>	Create each path segment; ignore <code>ResourceExistsError</code>

7.3 File operations

Function	SDK call
<code>list_paths(fs, prefix, max_items=200)</code>	<code>get_paths(recursive=False)</code>
<code>download_file_bytes(fs, path)</code>	<code>get_file_client().download_file().readall()</code>
Upload (inline)	<code>get_file_client(path).upload_data(bytes, overwrite=)</code>
<code>delete_file</code>	<code>get_file_client().delete_file()</code>
<code>delete_directory_recursive</code>	<code>get_directory_client().delete_directory()</code>

8. Upload flow

Upload writes the PDF to Azure Blob, records metadata in MySQL, and logs the blob audit entry.

Diagram 8.1 — Part A: Auth & validation

Diagram 8.1 — Part A: Auth & validation

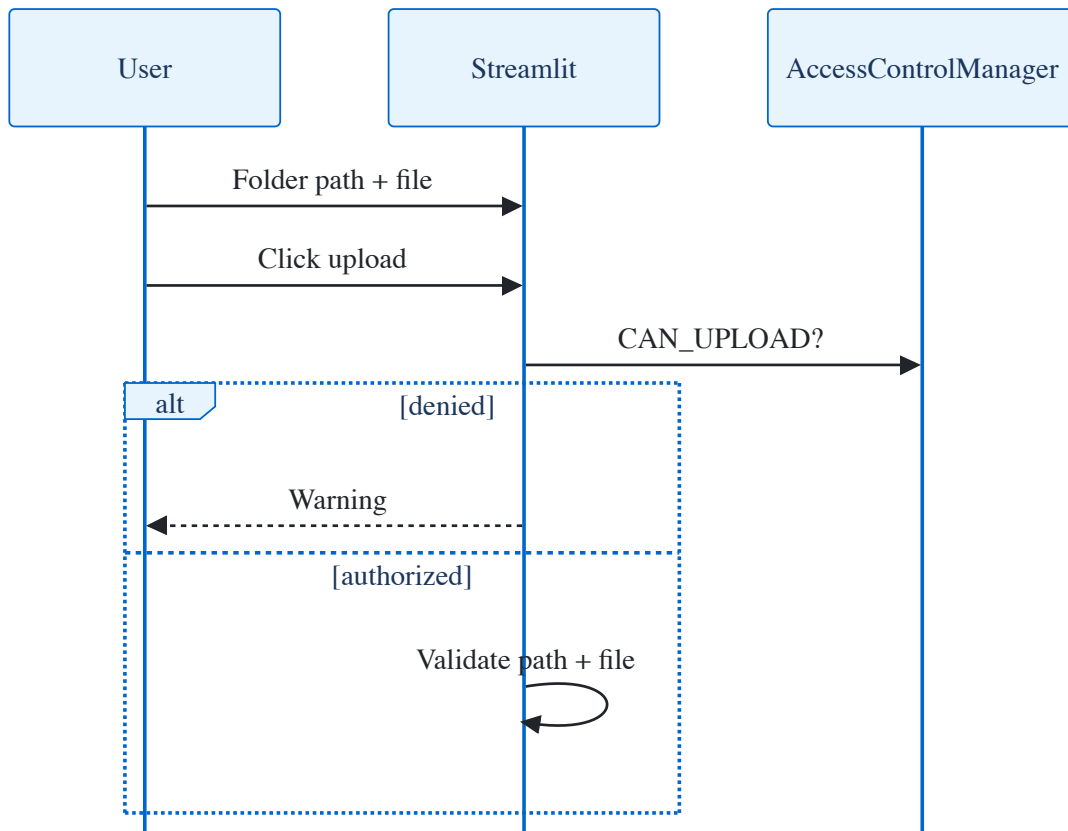
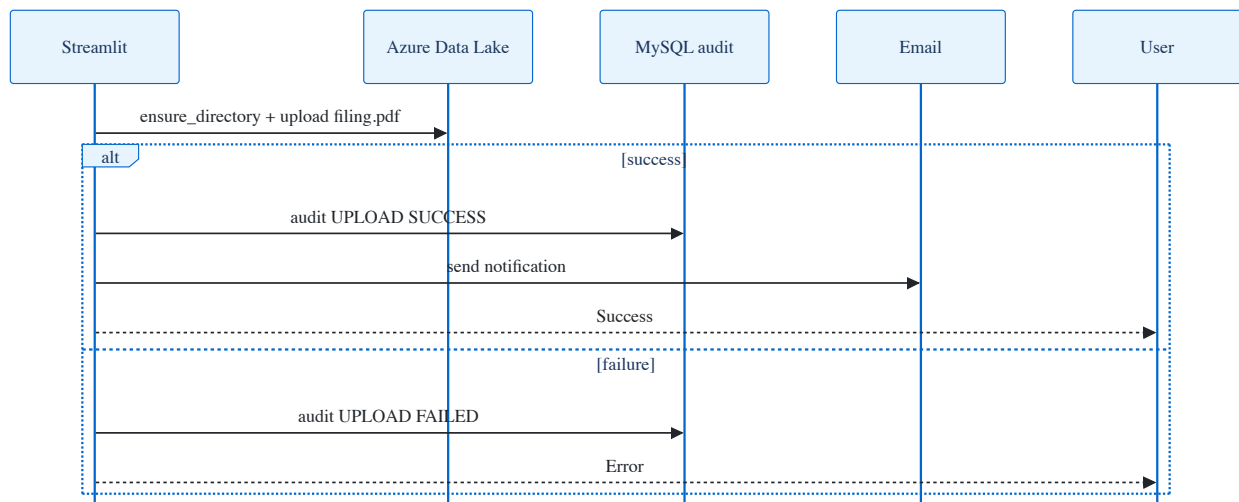


Diagram 8.1 — Part B: Azure write & audit

Diagram 8.1 — Part B: Azure write & audit



8.1 Upload path resolution example

User input	AZURE_BLOB_PREFIX	Resulting ADLS path
AAPL/2024/annual-report	(empty)	AAPL/2024/annual-report/filing.pdf
BRBY/2025/interim-report-Q1	filings	filings/BRBY/2025/interim-report-Q1/filing.pdf

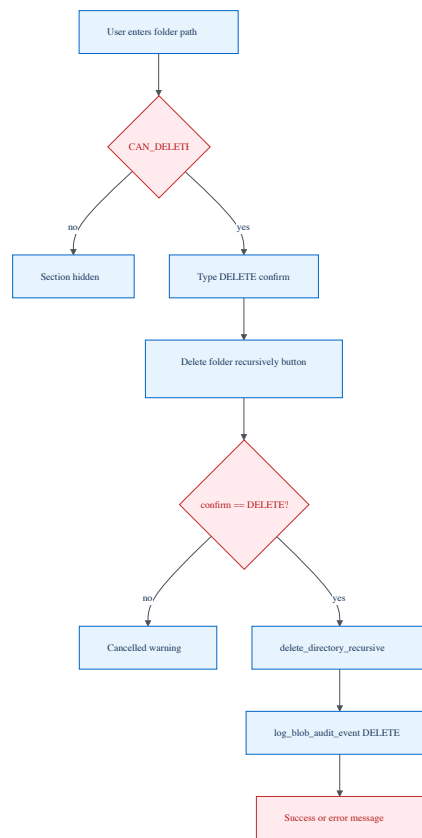
8.2 Email behavior

- **TO:** `UPLOAD_SUCCESS_RECIPIENTS` (normalized, deduplicated)
- **CC:** Logged-in uploader (unless already in TO)
- **Subject:** `Company Filings Upload Success - 1 file`
- **Body:** uploader email, container, folder, overwrite flag, original vs saved filename, byte size, azure path
- Email failure → `st.warning` but upload still succeeds

9. Delete flow

Delete removes the blob object, soft-deletes metadata, and records an audit log row.

Delete removes the blob object, soft-deletes metadata, and records an audit log row.



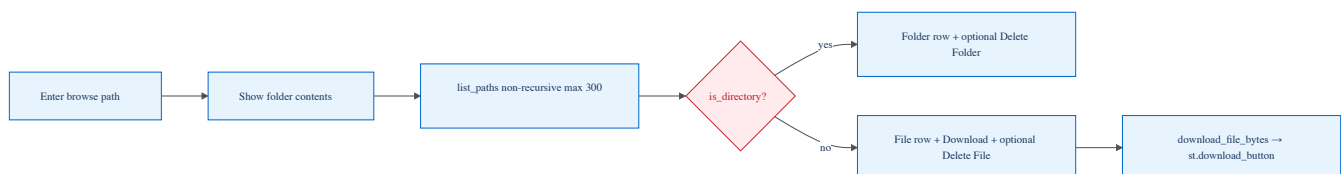
Browse-row deletes (single file or subfolder) follow the same audit pattern with `item_type` `FILE` or `FOLDER`.

`ensure_delete_access()` raises `PermissionError` if `CAN_DELETE` is false — defense in depth beyond UI hiding.

10. Browse / download flow

Browse lists non-SEC filings from the DB; download generates a time-limited blob SAS URL.

Browse lists non-SEC filings from the DB; download generates a time-limited blob SAS URL.



Downloads use `mime="application/octet-stream"` and basename as `file_name`.

11. Audit logging — `coreiq_azure_blob_audit_log`

11.1 Table bootstrap

`create_audit_table_if_not_exists()` runs on page load (`CREATE TABLE IF NOT EXISTS`).

Failure → `st.error` + `st.stop()`.

11.2 Schema

Column	Type	Description
<code>id</code>	BIGINT AUTO_INCREMENT	Primary key
<code>event_time</code>	DATETIME	Default <code>CURRENT_TIMESTAMP</code>
<code>user_email</code>	VARCHAR(255)	Actor
<code>action_type</code>	VARCHAR(50)	<code>UPLOAD</code> , <code>DELETE</code>
<code>item_type</code>	VARCHAR(50)	<code>FILE</code> , <code>FOLDER</code>
<code>container_name</code>	VARCHAR(255)	Filesystem/container
<code>root_prefix</code>	VARCHAR(500)	<code>AZURE_BLOB_PREFIX</code>
<code>target_path</code>	TEXT	Full ADLS path
<code>target_name</code>	VARCHAR(500)	Filename or folder leaf
<code>file_size_bytes</code>	BIGINT	Upload size (nullable)
<code>overwrite_flag</code>	TINYINT(1)	Upload overwrite
<code>status</code>	VARCHAR(50)	<code>SUCCESS</code> , <code>FAILED</code>
<code>error_message</code>	TEXT	Failure detail
<code>created_at</code>	DATETIME	Insert timestamp

Indexes: `idx_coreiq_blob_audit_time` , `idx_coreiq_blob_audit_user` ,
`idx_coreiq_blob_audit_action`

11.3 Failure isolation

`log_blob_audit_event()` catches all exceptions and shows `st.warning` — **audit failure never blocks** the primary Azure operation.

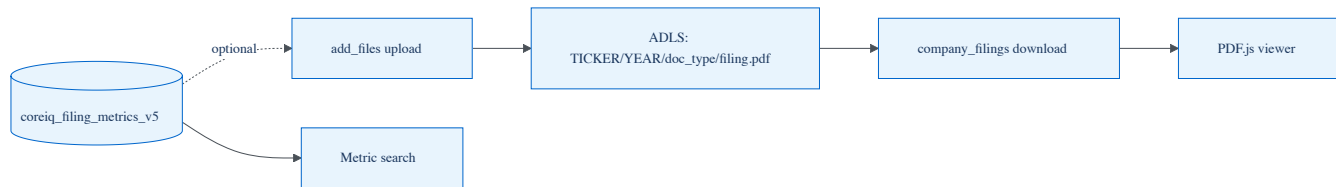
12. Components used

Module	Usage
<code>components.styles.hide_sidebar</code>	Sidebar hidden
<code>components.styles.render_styles</code>	Global CSS
<code>components.styles.set_page_layout</code>	Layout
<code>components.navigation.render_header</code>	"Company Filings File Manager"
<code>components.navigation.render_coresight_footer</code>	Footer
<code>core.auth_manager.get_current_user</code>	<code>LOGGED_IN_EMAIL</code>
<code>core.access_control.AccessControlManager</code>	Upload/delete gates
<code>core.config.config</code> , <code>Environment</code>	Staging/local guard + DB config
<code>utils.server_logger.new_rerun_id</code>	Log correlation

Not used: `app/utils/azure_blob.py` , `FilingMetricRepository` , `edgartools` , LLM extractor.

13. Relationship to Company Filings viewer

13. Relationship to Company Filings viewer



Concern	add_files page	company_filings page
SDK	<code>filedatalake</code>	<code>storage.blob</code>
Write	Yes	No
Read/list	Browse section	Viewer + scanner
Filename	Always <code>filing.pdf</code>	Expects <code>filing.pdf</code> (non-SEC) or <code>filing.html</code> (SEC)
Metric DB	No	Yes

Uploading via `add_files` **does not** insert rows into `coreiq_filing_metrics_v5`. Metric data requires separate ETL. The viewer can still show PDFs if blob exists; search may return empty until metrics are ingested.

14. Suggested folder conventions (non-SEC)

Align uploads with `VALID_DOC_TYPES` / `DOC_TYPE_TO_FOLDER` in `company_filings.py`:

Document	Example folder segment
Annual report	<code>{TICKER}/{YEAR}/annual-report</code>
Q1 interim	<code>{TICKER}/{YEAR}/interim-report-Q1</code>
Half-yearly	<code>{TICKER}/{YEAR}/half-yearly</code>

Example: `LULU/2025/interim-report-Q3` → ADLS file `.../LULU/2025/interim-report-Q3/filing.pdf`

15. Security considerations

1. **Path injection:** User-supplied folder paths passed to ADLS APIs — rely on Azure ACLs; no `..` sanitization in this module (unlike `filing_units.build_filing_blob_path`).
2. **Secrets:** Azure key and SMTP password from environment only.
3. **Production guard:** Page redirects away on prod `config.env`.
4. **Delete:** Recursive delete is destructive; gated by `delete_admin` + `DELETE` confirmation for bulk folder delete.
5. **Auth:** OIDC via `require_auth()` at page entry.

16. Error handling patterns

Scenario	User feedback
Azure connection fail on init	<code>st.error</code> + <code>st.stop()</code>
Audit table setup fail	<code>st.error</code> + <code>st.stop()</code>
Upload validation (no path/file)	<code>st.warning</code>
Upload Azure error	<code>st.error</code> + audit FAILED row
Audit write fail	<code>st.warning</code> (action completed)
Email fail	<code>st.warning</code> (upload succeeded)
Browse list fail	<code>st.error</code>
Delete permission	<code>PermissionError</code> → <code>st.error</code>

17. Function index

Function	Responsibility
<code>get_db_connection()</code>	PyMySQL with shared SSL config
<code>create_audit_table_if_not_exists()</code>	DDL for audit table
<code>log_blob_audit_event()</code>	Insert audit row (best-effort)
<code>get_datalake_clients()</code>	ADLS service + filesystem clients
<code>build_full_path()</code>	Prefix + user folder
<code>ensure_directory_exists()</code>	Recursive mkdir on ADLS
<code>list_paths()</code>	Non-recursive directory listing
<code>download_file_bytes()</code>	Full file read
<code>delete_directory_recursive()</code>	ACL-gated folder delete
<code>delete_file()</code>	ACL-gated file delete
<code>send_upload_success_email()</code>	SMTP TLS notification
<code>_normalize_email_list()</code>	Email list cleanup

18. Dependencies (Python packages)

Package	Usage
<code>streamlit</code>	UI framework
<code>azure-storage-file-datalake</code>	<code>DataLakeServiceClient</code>
<code>azure-core</code>	<code>ResourceExistsError</code>
<code>pymysql</code>	Audit DB writes
<code>python-dotenv</code>	Env loading
<code>stdlib</code> <code>smtplib</code> , <code>email.mime</code>	Upload notifications

19. Operational checklist

1. Confirm `config.env` is `LOCAL` or `STAGING` .
2. Set Azure ADLS credentials and container name.
3. Grant `edit` on `company_filings_add_files` for uploaders.
4. Grant `delete_admin` only to trusted admins.
5. Upload to paths matching `company_filings` non-SEC conventions.
6. Verify blob appears in Company Filings viewer (select matching ticker/year/doc type).
7. Run metric ETL separately if search coverage is required.